

• Events & Listeners :

Events هو عبارة عن حدث حصل عندى في التطبيق وليكن زى مستخدم عمل انشاء حساب او رفع صورة او غير كلمة مرور الخ

اما **Listeners** ده الاحداث اللى بعملها بعد حدث معين وليكن يوزر عمل تسجيل بعد عاوز يرسل رسالة على الايميل وكمان يعمل اشعار وكمان يسجل ده في log كل واحدة من دول تسمى **Listener**

بيتم انشاء **Events** من خلال الامر :

```
php artisan make:event UserRegistered // app/Events/UserRegistered.php
```

• ويتم انشاء **Listeners** من خلال الامر و يربطه بالحدث او **Event** :

```
php artisan make:listener SendWelcomeEmail --event=UserRegistered
```

```
1 class UserRegistered
2 {
3     public function __construct(
4         public User $user
5     ) {}
6 }
```

فقط ال **event** بمرر في البيانات بمثابة حاوية للبيانات بتاعتي **data Container**

اما **listener** ده اللى بنفذ في اللوجيك او المنطق بتاعى على الحدث او الايفنت بتاعى اللى ببعته للدالة **handle**

```
1
2 class SendWelcomeEmail
3 {
4     public function handle(UserRegistered $event)
5     {
6         Mail::to($event->user->email)
7             ->send(...);
8     }
9 }
```

ويتم تشغيل ال `event` داخل `controller` من خلال هذا الامر :

```
1 public function store()
2 {
3     $user = User::create(...);
4
5     UserRegistered::dispatch($user);
6 }
```

اول ما انفذ ميثود `dispatch()` لارفيل بتروح تشغل كل `Listeners` المرتبطة بالايفنت ده `UserRegistered`

: Event Discovery

في الاصدارات الحديثة من لارفيل لا تحتاج الى تسجيل `Listener` يدوياً لارفيل تبحث تلقائياً داخل فولدر `Listeners` واي كلاس يحتوى على ميثود في `handle` مبعوت فيها بارمتر كلاس ايفنت يتم تنفذه تلقائياً

وفى حالة لو هسجله يدوى هيتم تسجيله في ملف `App ServiceProvider`

```
1
2 use App\Domain\Orders\Events\UserRegistered;
3 use App\Domain\Orders\Listeners\SendWelcomeEmail;
4 use Illuminate\Support\Facades\Event;
5
6 /**
7  * Bootstrap any application services.
8  */
9 public function boot(): void
10 {
11     Event::listen(
12         UserRegistered::class,
13         SendWelcomeEmail::class,
14     );
15 }
```

: Queue Listener

أحيانا يكون في **Listener** بياخذ وقت طويل في التنفيذ وليكن ربط مع **api** او هرسل رسالة على الايميل او رسالة **sms** بدل مخرى المستخدم ينتظر العملية تخلص لا بخليها تنفذ في الخلفية مباشرة عن طريق **Queue** وبالتالي بخرى اداء الموقع افضل بكثير ويرجع الرد للمستخدم فوراً من غير منتظر العمليات تنفذ الاول انما هيرجع الرد للمستخدم وفي نفس الوقت شغال في الخلفية على العمليات ده من غير ميوتثر على اداء الموقع

❖ : Event Subscriber

ده بستخدم في حالات لو عندي عدد كبير من **Listener** بدل معمل لكل **Listener** كلاس خاص به بعمل كلاس **Event Subscriber** واحد بجمع في كل **Listener** بمعنى يحتوى على دوال كثير وكل دالة بتعمل حدث مختلف

في الحالة ده بيتنم انشاء كل **Events** من خلال الامر ده :

```
php artisan make:event UserRegistered
```

```
php artisan make:event UserUpdated
```

```
php artisan make:event UserDeleted
```

ثم بيتنم إنشاء **Subscriber** من خلال الامر ده :

```
php artisan make:listener UserSubscriber --subscriber
```

```
1 use App\Domain\Orders\Events\UserRegistered;
2 use App\Domain\Orders\Listeners\SendWelcomeEmail;
3 use Illuminate\Support\Facades\Event;
4
5 class UserSubscriber
6 {
7     public function handleRegister(UserRegistered $event)
8     {
9         // إرسال إيميل
10    }
11
12    public function handleUpdate(UserUpdated $event)
13    {
14        // تسجيل Log
15    }
16
17    public function handleDelete(UserDeleted $event)
18    {
19        // حذف ملفات المستخدم
20    }
21
22    public function subscribe($events)
23    {
24        $events->listen(
25            UserRegistered::class,
26            [UserRegistered::class, 'handleRegister']
27        );
28
29        $events->listen(
30            UserUpdated::class,
31            [self::class, 'handleUpdate']
32        );
33
34        $events->listen(
35            UserDeleted::class,
36            [self::class, 'handleDelete']
37        );
38    }
39 }
40
```

❖ Logging :

هو وظيفته تسجيل الاحداث والاطء التي تحدث في الموقع ولك لسهولة اكتشاف الاخطاء و مراقبة التطبيق و تتبع الاحداث

بدلا من كتابة `Echo or dd` لمعرفة ما يحدث سيتم تسجيل الحدث في `log` طبعا كل اعدادات ال `logging` ف ملف `config/logging.php`

لارفيل بتسجل الاحداث افتراضيا في `stack` وده عبارة عن `Channel` بمعنى هي المكان الذي ستذهب إليه الرسائل

❖ Available Channel Drivers :

لارفيل عنده قنوات كثير بيخزن فيها الرسائل بتاعته مثل :

١. Single :

يحفظ كل شيء في ملف واحد.

```
1 'single' => [  
2     'driver' => 'single',  
3     'path' => storage_path('logs/laravel.log'),  
4 ],
```

٢. Daily :

ينشئ ملفاً جديداً كل يوم.

```
1 'daily' => [  
2     'driver' => 'daily',  
3     'path' => storage_path('logs/laravel.log'),  
4     'days' => 14,  
5 ],
```

بعد ١٤ يوماً يحذف الملفات القديمة .

٣. Slack :

بدلاً من حفظ الخطأ فقط داخل ملف يرسل الأخطاء مباشرة إلى Slack مثل :
Server is down - Payment failed مفيد جداً في بيئة الإنتاج
.(Production)

٤. Syslog :

يرسل الرسائل إلى سجل الأحداث الخاص بنظام التشغيل يستخدم غالباً في Linux Servers .

٥. Errorlog :

يرسل الرسائل إلى PHP Error Log .

اما Stack ممكن يرسل الرسالة في اكثر من مكان

- ما هو Monolog ؟

لارفيل لا يكتب ال log بنفسه يعتمد على مكتبة اسمها Monolog وده اشهر مكتبة في php logging في

مستويات ال (Log Levels) :

- ١- Emergency :توقف كامل للتطبيق أو الخادم
- ٢- Alert :مشكلة تتطلب تدخلاً فورياً
- ٣- Critical : خطأ شديد
- ٤- Error : خطأ يمنع جزءاً من التطبيق من العمل
- ٥- Warning : تحذير، لكن التطبيق ما زال يعمل
- ٦- Notice : حدث غير اعتيادي لكنه ليس خطأ
- ٧- Info : معلومات عامة
- ٨- Debug : معلومات تفصيلية للمطور أثناء التطوير

وايضا يمكن اضافة Context بجانب البيانات

```
1 Log::info('Login failed', [  
2     'id' => $user->id,  
3     'email' => $user->email,  
4 ]);
```

• : Laravel Pail

هي عبارة عن أداء تعرض السجلات مباشرة أثناء تشغيل التطبيق للتثبيت باستخدام امر :

`composer require --dev laravel/pail`

وللتشغيل باستخدام امر : `php artisan pail`

• : Laravel Telescope

هي أداة رسمية من **Laravel** لمراقبة التطبيق أثناء التطوير

(Debugging & Monitoring) وهي عبارة عن لوحة تحكم تعرض كل ما يحدث داخل التطبيق، مثل الطلبات (Requests)، والاستعلامات (Queries)، والأخطاء (Exceptions)، والمهام (Jobs)، والبريد الإلكتروني (Mail)، وغيرها، مما يسهل اكتشاف المشاكل وتحسين الأداء.

يتم تثبيتها من خلال الامر ده : `composer require laravel/telescope --dev`

في الحالة ده بستخدمها في مرحلة التطوير فقط

: Data Pruning

Telescope يسجل كل شيء بمعنى في كل دقيقة ممكن يخرن الكثير من request , Queries وبالتالي بعد فترة جدول في الداتا بيز هياخد مساحة ضخمة جدا ولذلك يوجد امر لحذف البيانات القديمة

`php artisan telescope:prune`

ويتم تشغيله يوميا من خلال الاعدادات :

`Schedule::command('telescope:prune')->daily();`

افترضيا يحذف ماهو اقدم من ٢٤ ساعة

ما هي **Watchers** ؟

هو بمثابة مسئول عن نوع محدد من الاحداث ووظيفته بيقرب جزءا معيناً من التطبيق

مثل :

- **Query Watcher** وظيفته يعرض زمن الطلب وهل هو بطئاً ام لا .
- **Request Watcher** وظيفته يعرض (Response – Session – Cookies – Headers)
- **Mail Watcher** وظيفته يعمل معاينة كاملة سواء على المرفقات ومحتوى الرسالة وعنوانها

